ADR-0008 : Contrôle d'accès par chemins canoniques et autorité dérivée

- **Statut :** Accepté
- **Date :** 2026-10-27
- **Décideurs :** Driss OUAAZIZ — Software Architect

Contexte et problème

Le contrôle d'accès de FileFlow repose sur des **chemins virtuels** (relatifs à la racine de stockage) : chemins autorisés des groupes (`allowed_paths`), partages internes, dossiers partagés (collaboration), liens publics de partage (`/s/:token`) et demandes de dépôt public (`/r/:token`).

L'audit de sécurité d'octobre 2026 (document 14) a mis en évidence deux faiblesses de conception transversales, qu'aucun outil d'analyse statique ne détectait :

- **H1 — Comparaison de chemins non canonique.** Les vérifications de portée comparaient la chaîne brute de la requête par simple préfixe (`starts_with`). Un chemin tel que `/teamA/../teamB/secret` satisfaisait le préfixe `/teamA` tout en désignant, une fois résolu sur disque, une ressource hors du périmètre autorisé. De même, `/proj` couvrait à tort `/projects` .
- **Autorité figée à la création.** Plusieurs mécanismes de délégation conféraient des droits que leur créateur ne détenait pas, ou conservaient ces droits après leur retrait :
  - **H2** — un utilisateur en lecture seule pouvait créer une demande de dépôt public et obtenir ainsi un accès en écriture ;
  - **M1** — un dossier partagé accordait à ses membres des droits supérieurs à ceux de son créateur, et survivait à la révocation de ce dernier ;
  - **F5** — un lien public n'était pas rattaché au cycle de vie de la ressource : après un renommage ou une suppression, il pouvait exposer un **nouveau** fichier créé au même chemin, et il survivait au retrait des droits de son créateur.

Facteurs de décision

- Une seule représentation des chemins pour toutes les décisions d'autorisation.
- Comparaison de portée respectant les frontières de segments.
- Aucune délégation (partage, dossier partagé, lien, dépôt) ne doit excéder les droits propres de son créateur ; les délégations collectives ou anonymes (dossier partagé, lien, dépôt) ne doivent pas survivre au retrait de ces droits.
- Pas de nouvel état à synchroniser : la décision doit rester vraie quand les droits du créateur évoluent.
- Comportement en échec fermé (*fail closed*) sur les données héritées mal formées.

Options envisagées

1. **Filtrer les motifs dangereux (`..`) au cas par cas dans chaque gestionnaire** — Correctif local, fragile : chaque nouveau point d'entrée doit y penser, et la comparaison par préfixe brut reste fausse (`/proj` contre `/projects`).

2. **Comparer des chemins résolus sur disque (canonicalisation système)** — Dépend de l'existence de la ressource et des liens symboliques ; inadapté aux chemins persistés (partages, liens) qui doivent rester comparables même si la cible change.
3. **Figer les droits du créateur dans la délégation (instantané à la création)** — Simple, mais la délégation survit au retrait des droits du créateur ; c'est précisément le défaut constaté (M1, F5).
4. **Chemins virtuels canoniques, comparaison par segments et autorité dérivée réévaluée à chaque usage** *(retenu)*.

Décision

1. Chemins virtuels canoniques

- Toute vérification d'accès s'effectue sur la **forme canonique** du chemin, produite par `files::normalize_virtual` : barres multiples et segments `.` supprimés, chemin absolu commençant par `/`. Un segment `..` est **rejeté** (erreur 400) et non résolu ; les noms contenant simplement des points (`..b`, `c..`) restent valides.
- La portée d'un préfixe se teste par **segments** avec `files::path_covers` : `/proj` couvre `/proj` et `/proj/a/b`, mais pas `/projects`. C'est l'unique test de préfixe derrière les chemins de groupe, les partages internes et les dossiers partagés.
- Les `allowed_paths` des groupes sont validés et canonicalisés à l'écriture. Pour les entrées héritées, `files::canonical_prefix` n'honore que la graphie canonique (éventuellement suivie d'une barre finale) : une entrée mal formée (`.`, `//`, `teamA`, `/a//b`…) **n'accorde rien**, au lieu de s'élargir à la racine.
- Les chemins persistés (partages internes, dossiers partagés, liens publics, demandes de dépôt, favoris, métadonnées) sont stockés sous forme canonique.
- La création d'un droit ou d'un lien persistant sur une ressource atteinte via un lien symbolique est refusée (`files::ensure_direct_path`), afin que le chemin enregistré et le chemin réel restent cohérents lors des renommages et suppressions.

2. Autorité dérivée, vérifiée à chaque usage

Le droit **propre** d'un utilisateur (`files::own_grant`) se limite à son rôle, ses permissions résolues et les chemins de ses groupes, en **ignorant** les partages internes et les dossiers partagés. Toute délégation est bornée par ce droit propre, et ce contrôle est **réévalué à chaque accès**, pas seulement à la création :

DÉLÉGATION	À LA CRÉATION	À CHAQUE USAGE
Partage interne	`validate_own_access` (lecture, et écriture si accordée) : pas de re-partage transitif	Droit nominatif du destinataire, valable jusqu'à révocation du partage ou suppression de la ressource (droits de l'émetteur non réévalués)
Dossier partagé	`own_grant(FileRead)` ; accorder l'écriture à un membre exige `own_grant(FileWrite)`	Un membre n'obtient l'accès que si le créateur le détient encore en propre (`own_grant`)
Demande de dépôt public	`validate_own_access(FileWrite)` — un accès obtenu par partage interne ne suffit pas	Droit d'écriture du créateur revérifié à chaque dépôt ; sinon 404
Lien public de partage	`validate_access(FileRead)`	Créateur actif **et** toujours en lecture sur la ressource ; sinon 404 uniforme

Conséquence directe : **révoquer ou rétrograder le créateur révoque immédiatement ses dossiers partagés, liens publics et demandes de dépôt**, sans traitement de masse. Le partage interne, droit nominatif accordé à une personne désignée, reste en revanche borné à sa création seulement (voir risques résiduels).

3. Rattachement au cycle de vie de la ressource

- Lors d'un **renommage ou d'un déplacement**, les partages internes, dossiers partagés, favoris, liens publics et demandes de dépôt sont re-pointés vers le nouveau chemin (`rebase_path`).
- Lors d'une **suppression**, ces mêmes objets ainsi que les métadonnées (étiquettes, collections) sont purgés pour le chemin et sa descendance (`purge_under`). Un lien ne peut donc plus exposer un fichier recréé ultérieurement au même emplacement.
- Les dépôts publics n'écrasent jamais un fichier existant : le nom est réservé atomiquement (`create_new`) et numéroté en cas de collision (`rapport (1).txt`).

Conséquences

Positives

- Une seule définition du chemin et de la portée, testée unitairement et couverte par la cible de fuzzing `resolve` : la classe de contournement H1 est fermée pour tous les points d'entrée.
- Impossible de déléguer plus que ses droits propres ; dossiers partagés, liens publics et demandes de dépôt ne survivent pas au retrait de ces droits (H2, M1, F5).
- Réponses publiques uniformes (404) : un lien mort ne révèle pas la raison de son invalidité.
- Échec fermé sur les données héritées mal formées.

Négatives

- Coût d'évaluation supplémentaire à chaque accès par délégation (résolution des droits du créateur), négligeable au regard du cache mémoire des magasins (ADR-0001).
- Une rétrogradation du créateur interrompt sans préavis l'accès de ses destinataires et invalide ses liens ; c'est le comportement voulu, mais il peut surprendre les utilisateurs.
- Les chemins contenant `..` sont refusés plutôt que résolus : un client mal écrit reçoit une erreur 400.

Risques résiduels

- **Constat ouvert N5** : la création d'un lien public (`POST /api/shares`) n'exige que `validate_access(FileRead)` , qui inclut les partages internes. Un destinataire d'un partage interne en lecture peut donc publier un lien public sur la ressource reçue. Le lien reste borné par ses droits (et disparaît avec eux), mais il étend l'exposition au-delà des personnes choisies par le propriétaire. Arbitrage produit en attente (exiger `validate_own_access` , ou assumer ce comportement).
- Un partage interne n'est pas réévalué à l'usage contre les droits courants de son émetteur : il subsiste après une rétrogradation de celui-ci, jusqu'à sa révocation explicite ou la suppression de la ressource. À reconsidérer si l'on souhaite aligner ce mécanisme sur l'autorité dérivée.
- Tout nouveau point d'entrée doit passer par `normalize_virtual` et les fonctions de contrôle centralisées ; une matrice d'autorisation exécutable en CI est prévue au plan directeur des scans (document 15).

Références

- Document 14 — rapport d'audit de sécurité d'octobre 2026 (constats H1, H2, M1, F5) ; document 15 — plan directeur des scans et rapports de sécurité.

- Code : `backend/src/files.rs` (`normalize_virtual` , `path_covers` , `canonical_prefix` , `ensure_direct_path` , `validate_access` , `validate_own_access` , `own_grant` , `rename` , `move_items` , `delete`), `backend/src/{collab,internal_shares,shares,upload_requests}.rs` .
- OWASP — *Authorization Cheat Sheet* ; CWE-22 (*Path Traversal*) ; CWE-269 (*Improper Privilege Management*).
- ADR-0001 (persistance et cache), ADR-0002 (révocation des sessions).